

PRINCIPAL ENTERPRISE ARCHITECT REFERENCE SERIES

VOLUME 9 OF 10

Enterprise Reference Repository

A curated, practitioner-grade reference library: architecture patterns, anti-patterns, decision trees, ADR examples, governance policies, and Policy-as-Code templates — calibrated for banking and financial services environments.

Enterprise Architecture Review Board Handbook · Banking & Financial Services Edition · Continuation Volume

Part A — Reference Architectures & Architecture Patterns

The goal of this repository is practical reuse, not comprehensive coverage. Every entry has been selected because it represents a decision a Principal Architect in a banking context will genuinely face — not because it completes a theoretical taxonomy. Entries include what the pattern solves, its key trade-offs, when to avoid it, and a Mermaid diagram specification for version-controlled documentation.

USING MERMAID DIAGRAMS IN LIVING DOCUMENTATION

All diagrams in this volume are specified in `Mermaid` syntax, which can be rendered directly in Confluence, GitHub/GitLab, and most modern documentation platforms. This means the diagram lives as version-controlled text alongside ADRs and architecture records, rather than as a binary image file disconnected from its source — the living-documentation discipline from Volume 3, Section 5.8, applied concretely.

RA-01 — Event-Driven Core Banking Integration

Problem: Core banking platforms (Temenos, Finacle, FIS Profile, custom mainframe) are typically synchronous, monolithic, and intolerant of the fan-out integration demands that a modern digital bank imposes — mobile app, open banking APIs, real-time analytics, fraud detection, regulatory reporting, all needing to react to account events in near-real-time.

Solution: Introduce an event streaming backbone (Apache Kafka or cloud-native equivalent) as a mediator between the core and downstream consumers. The core emits domain events (AccountDebited, LoanStatusChanged, CustomerKYCUpdated) and each consumer subscribes independently — no direct coupling between consumers, and the core is insulated from demand spikes on any downstream system.

Trade-off	Detail
Gained	Decoupling; downstream scalability independent of core; natural audit trail (the event log itself is immutable); fan-out without core performance impact
Lost	Eventual consistency — consumers see events with a small lag, which is unacceptable for some use cases (e.g., real-time payment status checks by the customer must be handled with care; see RA-03 for CQRS complement)
Avoid when	The bank has very low event volume and the integration complexity of a broker outweighs its benefits; or when regulatory requirements demand synchronous confirmation of every downstream reaction to every event

```
graph LR
    CBS[Core Banking System] -->|Domain Events| KAFKA[Event Streaming Backbone]
    KAFKA -->|AccountDebited| FRAUD[Fraud Detection Service]
    KAFKA -->|AccountDebited| NOTIFY[Notification Service]
    KAFKA -->|AccountDebited| REPORTING[Regulatory Reporting]
    KAFKA -->|AccountDebited| ANALYTICS[Real-Time Analytics]
    KAFKA -->|LoanStatusChanged| CRM[CRM / Servicing Platform]
```

RA-02 — Strangler Fig for Core Banking Modernization

Problem: Replacing a legacy core banking platform in a single cutover (the "big bang" approach) carries catastrophic operational risk — the number of major bank outages attributable to failed big-bang cutovers is a significant data point — but allowing the legacy to persist indefinitely also has compounding costs.

Solution: The Strangler Fig pattern routes new capability requests to modern implementations while existing functionality remains on the legacy system, migrating capability-by-capability with each iteration gradually shrinking the legacy's footprint until it can be safely retired. The routing layer (often an API gateway or a specialized "façade" service) is the key architectural element.

Trade-off	Detail
Gained	Risk distributed across many small increments rather than concentrated in one cutover; ability to validate each migrated capability in production before the next; legacy system provides fallback at every stage
Lost	Extended dual-running cost — the bank runs both systems in parallel for the migration duration, which can be 3-7+ years for a full core banking migration; the routing facade itself becomes a technical debt item that must eventually be retired
Avoid when	The legacy system is so deeply intertwined that clean capability boundaries cannot be established without a foundational redesign first — in this case, a bounded decomposition exercise must precede the strangler pattern

RA-03 — CQRS for Regulatory Reporting at Scale

Problem: Regulatory reporting in banking (prudential, liquidity, AML, transaction monitoring) requires complex, time-consuming aggregations over large datasets that, if run against the transactional database, impose unacceptable performance impact on the write path — and the reporting data model is often radically different from the transactional one.

Solution: Command Query Responsibility Segregation (CQRS) separates the write model (transactional commands) from read models (optimized for regulatory reporting queries), with the read model updated asynchronously from the write model via the event stream from RA-01. Multiple specialized read models can coexist, each optimized for a different reporting shape.

Trade-off	Detail
Gained	Read model can be independently scaled; report queries no longer compete with payment processing for database resources; read models can be purpose-built for regulatory report schemas, removing complex runtime transformations
Lost	Eventual consistency on the read side (reports reflect data as of last event, not necessarily real-time); operational complexity of maintaining read model synchronization and reconciliation when events are missed or replayed
Avoid when	Reports must reflect exact-to-the-second transactional state — in this case the consistency cost of CQRS is unacceptable and a read replica of the transactional store may be more appropriate

RA-04 — Saga Pattern for Distributed Payment Flows

Problem: A payment initiation in a modern banking architecture typically spans multiple services (authorization, fraud check, AML screening, ledger posting, notification) with no single database that all services share. A failure in any step must not leave the system in a partially-completed state that can't be reconciled.

Solution: The Saga pattern manages a distributed multi-step transaction as a sequence of local transactions, each of which publishes an event consumed by the next step. If any step fails, compensating transactions are executed in reverse to undo prior steps' effects — preserving data consistency without a distributed transaction coordinator.

```
sequenceDiagram
    PaymentService->>FraudService: CheckFraud(paymentId)
    FraudService-->>PaymentService: FraudCheckPassed
    PaymentService->>AMLService: ScreenTransaction(paymentId)
    AMLService-->>PaymentService: AMLClear
    PaymentService->>LedgerService: PostDebit(accountId, amount)
    LedgerService-->>PaymentService: DebitPosted
    PaymentService->>NotifyService: SendConfirmation(customerId)
    Note over PaymentService: On failure at any step, compensating txns reverse prior steps
```

RA-05 — API Gateway with Backend-for-Frontend (BFF) for Omnichannel Banking

Problem: Mobile banking, web banking, branch systems, and open-banking third-party channels all consume overlapping but not identical data, at different payload sizes, with different latency tolerances and authentication requirements. A single generic API layer forces every channel to work around mismatches between their needs and the API's shape.

Solution: An API gateway handles cross-cutting concerns (authentication, rate limiting, SSL termination) while channel-specific Backend-for-Frontend services handle the composition, transformation, and aggregation each channel needs — the mobile BFF returns a lightweight, mobile-optimized payload; the web BFF returns a richer dataset; the open-banking BFF enforces regulatory protocol requirements (PSD2-equivalent).

RA-06 — Zero-Trust Network Architecture for Payment Processing Environments

Problem: Traditional perimeter-based security assumes that traffic inside the network boundary is trustworthy — a model that has been demonstrated repeatedly as insufficient in modern banking environments where sophisticated adversaries routinely achieve perimeter access and then move laterally with minimal additional friction.

Solution: Zero-trust treats every connection request as potentially hostile regardless of origin, requiring explicit verification (identity, device posture, intent) for every access request, minimum-privilege authorization scoped to the specific action requested, and continuous monitoring of in-session behavior rather than a one-time authentication event.

RA-07 — AI Model Gateway / Router Pattern

Problem: An enterprise banking organization deploying AI features across multiple products faces vendor concentration risk (dependence on a single AI provider), cost optimization opportunities (routing simple queries to cheaper models), and resilience requirements (failover if a provider has an outage) — none of which are well-handled by direct, point-to-point integration with a single AI model API in each product.

Solution: A model gateway sits between consuming services and AI model providers, handling routing, rate limiting, cost tracking, provider failover, and response caching. A routing policy engine directs queries based on complexity, cost envelope, latency requirements, and data-sensitivity classification (some queries must not leave approved providers due to data residency requirements).

Trade-off	Detail
Gained	Provider portability; cost optimization through tiered routing; operational visibility into AI spend and usage patterns; single point for audit logging of all AI interactions
Lost	An additional network hop adds latency and represents a new single point of failure that must itself be made highly available; routing logic adds implementation complexity; the gateway must be updated when new models or providers are adopted
Avoid when	AI adoption is still exploratory and a single provider relationship suffices for the near term — the gateway's benefits scale with provider diversity and query volume

```
graph TD
    APP1[Customer Service App] -->|Query| GW[AI Model Gateway]
    APP2[Fraud Scoring Service] -->|Query| GW
    APP3[Document Extraction] -->|Query| GW
    GW -->|Simple/cheap queries| GPT_MINI[Small Model - Low Cost]
    GW -->|Complex queries| GPT4[Large Model - High Capability]
    GW -->|Sensitive data - on-prem requirement| ONPREM[Self-Hosted Model]
    GW -->|Failover| BACKUP[Backup Provider]
    GW -->|Cost + Usage + Audit| OBSERVABILITY[Observability Platform]
```

RA-08 — Customer 360 Data Product

Problem: Customer data in a typical large bank is fragmented across dozens of systems — core banking, CRM, loan origination, mobile app, card systems, wealth management — with no single, curated, authoritative view. Downstream systems (fraud detection, personalization, regulatory reporting, customer service) each maintain their own partial views, creating inconsistency, duplication, and significant data quality risk.

Solution: A Customer 360 data product (Volume 3, Part B) aggregates, reconciles, and exposes a curated, versioned, SLA-backed customer view as a reusable data product rather than a point-to-point integration. Ownership, quality, and freshness are explicit — the product has a named owner, a defined quality contract, and observable freshness metrics.

RA-09 — Agentic AI with Human-in-the-Loop for KYC Remediation

Problem: KYC (Know Your Customer) remediation — identifying and resolving gaps in customer due diligence records — is a labor-intensive process typically handled by operations teams manually reviewing thousands of records against document and data requirements. It is highly repetitive, yet the consequences of errors (regulatory exposure, financial crime risk) are severe enough that full automation without oversight is inappropriate.

Solution: An AI agent handles the high-volume, well-defined portion of the remediation workflow (identifying gaps, retrieving available data, drafting outreach to customers, pre-populating forms) with human reviewers handling exception cases and final authorization decisions. The agent specification (Volume 4, Section 7.5) explicitly defines the boundary: agent acts autonomously up to drafting/staging; human approves before any customer-facing action or record modification.

AP-01 through AP-15 — Architecture Pattern Catalog

ID	Pattern	Problem solved	Key trade-off
AP-01	Circuit Breaker	Prevents cascading failure when a downstream service is degraded	Adds false-positive risk: circuit opens under transient load and blocks valid traffic
AP-02	Bulkhead	Isolates resource pools so failure in one consumer doesn't exhaust resources for others	Adds resource overhead: dedicated pools are less efficient than shared
AP-03	Retry with Exponential Backoff	Recovers from transient failures without overloading the failing service	Retry storms under correlated failure; must combine with jitter
AP-04	Idempotency Keys	Makes payment and state-changing operations safe to retry without double-processing	Requires idempotency store with appropriate retention; adds latency for key lookup
AP-05	Outbox Pattern	Guarantees exactly-once event publishing even when the event broker is temporarily unavailable	Polling overhead; outbox table becomes a bottleneck at very high throughput
AP-06	Event Sourcing	Preserves the full history of state changes as an immutable event log — strong audit trail	Query complexity; event schema evolution is hard; eventual consistency for read models
AP-07	Sidecar	Attaches operational concerns (logging, mTLS, configuration) to service instances without modifying service code	Adds per-instance resource overhead; complexity of keeping sidecar and service in sync
AP-08	API Versioning via Content Negotiation	Allows API evolution without breaking existing consumers	Multiple active versions create maintenance burden; requires disciplined consumer migration strategy
AP-09	Data Mesh / Domain Ownership	Shifts data product ownership to domain teams, improving quality accountability	Federated ownership increases data governance coordination cost; risk of inconsistent standards
AP-10	RBAC + ABAC Hybrid Authorization	Role-based access for coarse-grained control, attribute-based for fine-grained (e.g., access rules varying by customer segment or product)	Policy complexity grows quickly; authorization service becomes a critical path dependency
AP-11	Feature Flags / Dark Launching	Decouples deployment from release; enables gradual rollout and instant rollback	Technical debt accumulates if flags are not cleaned up; flag sprawl makes behavior hard to reason about
AP-12	Prompt Caching	Reduces LLM API cost for calls sharing a common system prompt by caching the prompt context server-side	Only effective when prompt prefix is stable; cached context can become stale if the cached portion contains dynamic content
AP-13	RAG (Retrieval-Augmented Generation)	Grounds LLM responses in enterprise knowledge without fine-tuning; enables up-to-date answers from a curated knowledge base	Retrieval quality ceiling determines answer quality; hallucination risk on retrieved context; access control on retrieval must mirror source system controls
AP-14	Competing Consumers	Scales event/message processing horizontally by running multiple consumer instances against a single queue	Message ordering not guaranteed across consumers; requires idempotent processing (AP-04)
AP-15	Transactional Outbox + Inbox	Implements reliable exactly-once message delivery end-to-end across producer and consumer without a distributed transaction	Two additional tables and polling overhead; latency marginally higher than at-most-once delivery

ANT-01 through ANT-15 — Anti-Pattern Catalog

ID	Anti-Pattern	How it manifests	Root cause & remedy
ANT-01	Distributed Monolith	Services are deployed separately but share a database or call each other synchronously for every operation, giving the worst of both worlds: deployment complexity without loose coupling	Missing domain boundary design; remedy is proper bounded context decomposition before decomposing deployment
ANT-02	Chatty Microservices	A single business operation requires 15+ synchronous inter-service calls, creating compounding latency and cascading failure risk	Insufficient co-location of highly cohesive operations; apply event choreography or aggregate coarser-grained services
ANT-03	Shared Mutable Database	Multiple services write to a single shared schema, creating implicit coupling through the database layer that defeats the purpose of service boundaries	Short-cut taken to avoid API design effort; each service must own its data and expose it through a defined interface
ANT-04	God Service	One service has accumulated so many responsibilities that it is functionally a monolith deployed as a "service," making it the most-changed, most-fragile, most-coupled component in the system	Missing ownership boundaries; apply strangler fig (RA-02) to decompose, starting with the highest-churn capabilities
ANT-05	Callback Hell / Event Spaghetti	Event-driven architecture where the event flow is so complex and non-obvious that no single person can trace a business process end-to-end through the events	Events introduced without flow documentation; remedy is explicit event catalog with choreography diagrams and fitness functions (Vol 2 S4.4) validating event flow invariants
ANT-06	Accidental Eventual Consistency	A system is built as eventually consistent because the team used an event broker, not because eventual consistency was a deliberate design choice — and the use cases require stronger consistency	Consistency requirements not established in Quality Attribute Workshop (Vol 2 S4.5) before design; fix by establishing the consistency model required per use case, then choosing architecture accordingly
ANT-07	The Mega-ADR	An Architecture Decision Record that documents a single, sweeping design decision covering dozens of choices in one document — so large it's never read and never updated	ADRs should cover one decision; if a design involves many decisions, each gets its own ADR, cross-referenced
ANT-08	Security Through Obscurity	API endpoints or internal services "protected" by not being documented or advertised, with no actual authentication or authorization enforcement	Time/cost pressure and the assumption that internal network = trusted; zero-trust (RA-06) and explicit auth enforcement on every endpoint
ANT-09	Snowflake Infrastructure	Each deployment environment is hand-configured uniquely, so no two environments are actually equivalent — "it works in staging" becomes meaningless as a test signal	Infrastructure-as-code with strict environment parity enforcement and automated drift detection
ANT-10	Vendor-Mediated Integration	Two internal systems integrated not directly but through a third-party vendor's platform that neither team controls, creating a governance and performance bottleneck	Often a legacy of individual vendor relationship decisions; remedy is event-driven direct integration where feasible, or explicit vendor-dependency risk management where not
ANT-11	Prompt Injection Blindness	An AI agent that processes external input (emails, documents, web content) does so without any sanitization or sandboxing, allowing an attacker to inject instructions that redirect the agent's behavior	AI-specific threat modeling (Volume 4 S7.2) not performed; remedy is explicit input sanitization, agent tool permission scoping to minimum necessary, and output validation before action
ANT-12	Model Monoculture	Enterprise-wide AI deployment entirely dependent on a single model provider, with no fallback or alternative routing — full outage if the provider has a disruption	AI Model Gateway (RA-07) with multi-provider routing and tested failover
ANT-13	Unbounded Agent Autonomy	An AI agent deployed with write access to production systems and no explicit scope-of-action limits — the agent can do anything its available tools allow, without per-action authorization	Missing Agent Specification (Volume 4 S7.5); remedy is explicit tool permission scoping and human-in-the-loop gates for high-impact actions
ANT-14	The Living Dead System	A system officially classified as "in decommission" for 3+ years that never actually gets decommissioned because new dependencies keep appearing — active technical debt masquerading as an in-progress retirement	No Retirement Checklist (Volume 4 S7.6) with dependency verification; no ARB gate requiring decommission sign-off; no one owns the closure
ANT-15	Cost-Blind Architecture	Architecture designed for technical elegance without modeling the economics at scale — discovered to be prohibitively expensive only after deployment, at which point rearchitecting is painful	Architecture economics (Volume 2) not applied at design time; remedy is mandatory cost-at-scale projection as part of the ARB submission form (Volume 8 S15.5)

Part B — Decision Trees, ADR Examples & Policy-as-Code Library

DT-01 — Integration Pattern Selection

```
flowchart TD
    A[New integration needed] --> B{Near real-time required?}
    B -->|Yes| C{Source system event-capable?}
    C -->|Yes| D[Event Streaming - AP-01/RA-01]
    C -->|No| E{Tolerate up to 5min lag?}
    E -->|Yes| F[Change Data Capture CDC]
    E -->|No| G[Polling API with short interval + caching]
    B -->|No| H{Volume > 10M records/batch?}
    H -->|Yes| I[Bulk ETL with incremental delta loading]
    H -->|No| J{Bidirectional sync required?}
    J -->|Yes| K[Event sourcing or operational data store]
    J -->|No| L[Scheduled API pull or file transfer]
```

DT-02 — Cloud Deployment Model Selection

```
flowchart TD
    A[New cloud deployment decision] --> B{Data residency constraint?}
    B -->|Specific country only| C{Approved cloud region exists?}
    C -->|Yes| D[Public cloud, constrained region]
    C -->|No| E[Private cloud or sovereign cloud]
    B -->|Flexible| F{Regulatory classification?}
    F -->|PCI/payment sensitive| G{On-prem latency required?}
    G -->|Yes| H[On-prem or private cloud + PCI zone]
    G -->|No| I[Public cloud, dedicated PCI landing zone]
    F -->|Standard confidential| J{Multi-cloud strategy active?}
    J -->|Yes| K[Evaluate primary + secondary provider fit]
    J -->|No| L[Enterprise standard cloud provider]
```

DT-03 — AI Use Case Risk Tier Assignment

```
flowchart TD
    A[New AI use case proposed] --> B{Does it make or materially influence decisions affecting customers?}
    B -->|Yes| C{Credit, insurance, employment, housing related?}
    C -->|Yes| D[HIGH RISK - Full ATAM + Model Risk + RAI Council + AI Gov Board]
    C -->|No| E{Does it generate customer-facing communications?}
    E -->|Yes| F[MEDIUM-HIGH - AI Gov Board + RAI Council + ARB review]
    E -->|No| G[MEDIUM - AI Gov Board light review + ARB]
    B -->|No| H{Does it take autonomous actions in production systems?}
    H -->|Yes| I{Actions reversible within 24hr?}
    I -->|No| J[HIGH RISK - Full review including Agent Spec mandatory]
    I -->|Yes| K[MEDIUM - Agent Spec + ARB + AI Gov Board]
    H -->|No| L{Internal tool only, no customer data?}
    L -->|Yes| M[LOW - Architecture CoP review sufficient]
    L -->|No| N[MEDIUM - Standard ARB review]
```

DT-04 — Build vs. Buy Decision

```
flowchart TD
    A[Capability needed] --> B{Is this differentiating to our competitive position?}
    B -->|Yes| C{Do we have the engineering talent to build and sustain it?}
    C -->|Yes| D[BUILD - with explicit make-to-buy reassessment trigger]
    C -->|No| E[BUY - with exit clause and future internalization plan]
    B -->|No| F{Is there a vendor solution meeting regulatory fit?}
    F -->|No| G[BUILD - regulatory constraint forces it]
    F -->|Yes| H{Vendor concentration risk acceptable?}
    H -->|No| I[BUILD or multi-vendor with abstraction layer]
    H -->|Yes| J{10yr TCO favors buy?}
    J -->|Yes| K[BUY - with contract review for lock-in clauses]
    J -->|No| L[BUILD or hybrid - economics favor internal]
```

DT-05 — Governance Routing (Which Review Path?)

```
flowchart TD
    A[New initiative] --> B{Does it follow an approved golden path exactly?}
    B -->|Yes| C[Platform-first - automated gate only, no ARB queue]
    B -->|No| D{Budget or team impact threshold exceeded?}
    D -->|No| E{New technology / pattern not in catalog?}
    D -->|Yes| F[Domain ARB - fast-track review]
    E -->|Yes| G[Central ARB - standard review]
    E -->|No| H{Cross-domain or cross-BU impact?}
    H -->|Yes| I[Central ARB + Executive Steering Committee brief]
    H -->|No| J{AI component present?}
    J -->|Yes| K[AI Solution Review - combined parallel gate]
    J -->|No| L[Standard Central ARB review]
```

ADR Examples

ADR-001 — ADOPT APACHE KAFKA AS ENTERPRISE EVENT STREAMING BACKBONE

Date: [Date] | **Status:** Accepted | **Deciders:** Chief Architect, Domain Architects, CTO Council

Context: The enterprise currently uses point-to-point synchronous integration between core banking, mobile banking, fraud detection, and regulatory reporting systems. This creates tight coupling, limits downstream system scale independence, and requires every new integration to be separately negotiated with the core banking platform team. We need an integration backbone that decouples producers from consumers, supports replay for new consumer onboarding, and provides a durable audit log of domain events.

Decision: We will adopt Apache Kafka (managed via [cloud provider] MSK / Confluent Cloud) as the enterprise event streaming backbone for domain events. All teams producing or consuming domain events will do so through this backbone rather than point-to-point.

Options considered:

Option A: Kafka/Confluent Cloud — strong ecosystem, proven at banking scale, managed offering reduces operational burden. Selected.

Option B: [Cloud provider native] — lower operational overhead, tighter cloud integration, but higher vendor lock-in risk given multi-cloud strategy.

Option C: RabbitMQ — mature, simpler, but lacks Kafka's log retention/replay model which is required for new consumer onboarding and audit trail use cases.

Consequences — positive: Decoupling of core banking from downstream consumers; replay capability for new consumers; durable audit log; scales downstream systems independently.

Consequences — negative: Kafka operational expertise required; introduces eventual consistency that teams must design for explicitly; a new critical infrastructure component requiring high-availability design and operational runbooks.

Compliance notes: Event data at rest will be encrypted; PII in events must be tokenized per Data Governance policy; event retention periods set per regulatory requirement by topic classification.

ADR-002 — USE CQRS FOR REGULATORY REPORTING READ MODELS

Date: [Date] | **Status:** Accepted | **Deciders:** Domain Architect (Regulatory Reporting), Data Architect

Context: Regulatory reporting queries against the transactional database are causing performance degradation during month-end/quarter-end processing windows. The regulatory report data model is materially different from the transactional model, and each report type requires different aggregations. Adding indexes to the transactional database creates DML slowdowns on the payment processing write path.

Decision: Implement CQRS with purpose-built read models for the three highest-volume regulatory report types (LCR, NSFR, and AML transaction monitoring feeds). Read models will be maintained via consumption of domain events from the Kafka backbone (ADR-001).

Consequences — positive: Zero contention with transactional write path; read models optimized for their specific report schema; can independently scale report-generation infrastructure during peak periods.

Consequences — negative: Eventual consistency — reports reflect data as of last event processing, with SLA of [X minutes] lag. Reconciliation process required for end-of-day regulatory submissions to validate read model completeness. Additional operational complexity for read model rebuild on schema change.

ADR-003 — AGENT SPECIFICATION: KYC REMEDIATION AGENT SCOPE OF AUTONOMY

Date: [Date] | **Status:** Accepted | **Deciders:** AI Solution Architect, Compliance Lead, ARB

Context: The KYC Remediation Agent (RA-09) requires a formally documented scope of autonomy as a governance prerequisite for ARB and AI Governance Board approval. This ADR captures the authority boundaries that the Agent Specification (Volume 4 Section 7.5) formalizes for this specific agent.

Decision — Permitted autonomous actions (no human approval required per action):

- Read-only access to customer record system to identify KYC gaps
- Cross-reference external data sources (credit bureau, sanctions lists) for gap resolution candidates
- Draft remediation action items and customer outreach messages (staging only, not sending)
- Categorize customers into remediation priority tiers

Decision — Requires human approval before execution:

- Any customer-facing communication
- Any write to the customer record system
- Any recommendation to exit a customer relationship (flag for compliance review)
- Any case above defined complexity threshold (agent confidence score below 0.75 on remediation path)

Consequences: Human review remains in the critical path for all consequential actions, preserving regulatory accountability. Agent delivers value through throughput improvement on the screening and preparation steps, which represent approximately 70% of current operations team time per case.

ADR-004 — REJECT GRAPHQL FEDERATION IN FAVOUR OF REST BFF FOR OPEN BANKING

Date: [Date] | **Status:** Accepted (option rejected documented here)

Context: The open banking channel team proposed adopting GraphQL Federation as the API layer for third-party provider (TPP) integration under PSD2-equivalent regulation. The proposal argued that GraphQL's flexibility would reduce per-TPP integration cost.

Decision: Rejected. Adopt REST BFF pattern (RA-05) instead.

Rationale for rejection:

1. Open banking regulatory standards (PSD2 and successors) specify REST/OpenAPI as the required interface standard. GraphQL does not meet this regulatory interface requirement.
2. Introspection of GraphQL schemas exposes internal data model details to third parties beyond what is required or appropriate.
3. Rate limiting and cost-per-query are materially harder to enforce precisely on GraphQL versus REST, which is important for TPP access tier enforcement.

This ADR is recorded to prevent re-proposal without new information addressing these constraints, which have been recurring in informal design discussions.

Policy-as-Code Library — Banking Starter Templates

```
# PAC-001: No direct cross-domain database access
# Platform: OPA (Open Policy Agent)
# Enforcement: CI/CD pipeline + runtime admission controller

package banking.architecture.cross_domain

import future.keywords.if
import future.keywords.contains

deny contains msg if {
    connection := input.service_connections[_]
    connection.type == "database"
    connection.source_domain != connection.target_domain
    not connection.api_mediated == true
    msg := sprintf(
        "Service '%v' (domain: %v) has direct database access to domain '%v'. Cross-domain data access must go through the owning domain's API or event stream.",
        [connection.source_service, connection.source_domain, connection.target_domain]
    )
}
```

```

# PAC-002: Encryption at rest required for classified data stores
# Platform: Terraform Sentinel / OPA for IaC
# Enforcement: Pre-apply in Terraform pipeline

policy "data_at_rest_encryption" {
  rule "storage_must_be_encrypted" {
    all tfplan.resource_changes as _, changes {
      changes.type in [
        "aws_s3_bucket", "azurerm_storage_account",
        "google_storage_bucket", "aws_rds_instance"
      ] and changes.change.after.tags.data_classification in [
        "confidential", "restricted", "pii"
      ] implies (
        changes.change.after.server_side_encryption_configuration != null or
        changes.change.after.encryption != null
      )
    }
  }
}

```

```

# PAC-003: No public S3 buckets for any classification
# Platform: AWS Config Rule / OPA
# Enforcement: Continuous compliance monitoring

package banking.cloud.s3_public_access

deny[msg] {
  resource := input.resource
  resource.type == "aws_s3_bucket"
  resource.properties.public_access_block_configuration.block_public_acls == false
  msg := sprintf(
    "S3 bucket '%v' does not have public ACLs blocked. All S3 buckets in the banking environment must block public access
    regardless of data classification.",
    [resource.properties.bucket_name]
  )
}

```

```

# PAC-004: AI agent tool permissions must not include ledger write access
#           without an approved Agent Specification ADR reference
# Platform: Custom admission controller / OPA
# Enforcement: Deployment gate

package banking.ai.agent_permissions

deny[msg] {
  agent := input.agent_deployment
  tool := agent.tools[_]
  tool.name contains "ledger"
  tool.permission contains "write"
  not agent.metadata.approved_agent_spec_adr
  msg := sprintf(
    "Agent '%v' requests write access to ledger tool '%v' but has no approved Agent Specification ADR reference. All ledger write
    permissions require a formally approved Agent Specification (Volume 4 Section 7.5).",
    [agent.name, tool.name]
  )
}

```

```

# PAC-005: Production deployment requires vulnerability scan completion
# Platform: GitHub Actions / Jenkins pipeline gate
# Enforcement: CD pipeline pre-deploy check

name: vulnerability-gate
on: [deployment]

jobs:
  security-gate:
    runs-on: ubuntu-latest
    steps:
      - name: Check vulnerability scan results
        run: |
          CRITICAL_COUNT=$(cat scan-results.json | jq '.vulnerabilities | map(select(.severity == "CRITICAL")) | length')
          HIGH_COUNT=$(cat scan-results.json | jq '.vulnerabilities | map(select(.severity == "HIGH" and .age_days > 30)) | length')
          if [ "$CRITICAL_COUNT" -gt "0" ]; then
            echo "BLOCKED: $CRITICAL_COUNT critical vulnerabilities. Remediate before deployment."
            exit 1
          fi
          if [ "$HIGH_COUNT" -gt "0" ]; then
            echo "BLOCKED: $HIGH_COUNT high vulnerabilities older than 30 days. Remediate or obtain ARB exception."
            exit 1
          fi
          echo "Vulnerability gate passed."

```

```

# PAC-006: Cost alert — resource spend growth threshold
# Platform: AWS Cost Anomaly Detection / Azure Cost Alerts (Terraform)
# Enforcement: Continuous monitoring

resource "aws_ce_anomaly_monitor" "architecture_cost_gate" {
  name           = "per-service-spend-anomaly-monitor"
  monitor_type   = "DIMENSIONAL"
  monitor_dimension = "SERVICE"
}

resource "aws_ce_anomaly_subscription" "spend_alert" {
  name           = "architecture-spend-alert"
  threshold      = 20    # Alert if spend is 20% above expected
  frequency      = "DAILY"
  monitor_arn_list = [aws_ce_anomaly_monitor.architecture_cost_gate.arn]
  subscriber {
    address = var.finops_alert_email
    type    = "EMAIL"
  }
}

```